

agent-cage vs Journeyman

Competitive analysis — what to build better, why Journeyman wins, and the recommended path

2026-06-19 · pnnl/agent-cage vs @journeyman/sandbox

BOTTOM LINE

They aren't competitors on the same axis. Journeyman owns **compute isolation + orchestration**; agent-cage owns the one thing we lack — **network egress governance**. Don't adopt their prototype. **Build a native egress layer** into our Docker backend, borrowing their proven proxy + allowlist + audit pattern.

The two axes that matter

Every agent-sandbox product sits on two independent axes. We win one; agent-cage wins the other — and the other is our gap.

1 · Compute isolation

where the agent runs

Journeyman — strong

agent-cage — basic

Per-run containers, named volumes, CPU/mem limits, pluggable backends, managed image builds.

2 · Egress governance

what the agent can reach

Journeyman — missing

agent-cage — strong

Domain allow/deny, HTTP-method filter, TLS inspection, per-request audit log.

The gap, proven from our code

Our entire network model is one line in `docker-execution-environment.ts:52`:

```
const network = config["network"] === "none"
  ? ("none")
```

```
: (spec.network ?? ("full")); // default = full
```

When `"full"`, no `NetworkMode` is set, so the container joins the default bridge with **unrestricted outbound internet**. The model is binary, and the default is the open end:

SETTING	BEHAVIOR	CONSEQUENCE
<code>"full"</code> <code>default</code>	Unrestricted egress to the whole internet	Agent can exfiltrate code/secrets anywhere. No record of what it contacted.
<code>"none"</code>	Loopback only	Breaks pip / npm / apt / git — box becomes useless.

A full grep for `HTTP_PROXY` / proxy / CoreDNS / allowlist across sandbox, agent-runtime and core returns nothing. There is no middle ground — exactly the gap agent-cage exists to fill.

Head-to-head

DIMENSION	JOURNEYMAN	AGENT-CAGE
Primary strength	Compute isolation + orchestration	Network egress governance
Backends	local, docker, windows (+planned ecs/k8s)	single Docker container
Per-run isolation	container + named volume per run	one shared container
Resource limits	CPU + memory	none documented
Network model	<code>binary on/off</code>	<code>allowlist + method filter</code>
Egress audit	none	full per-request log
Image lifecycle	build loop, fingerprint, lease concurrency	none
Multi-tenancy	org/system scope, many concurrent runs	single-operator
Durable execution	Conductor, retries, pause/resume	none
Maturity	production-pattern	self-declared research prototype

Why Journeyman is better

- **It's a platform, not a demo.** Multi-backend, multi-tenant orchestration with durable execution vs one container + a dashboard.
- **Real per-run isolation.** Each run gets its own container + volume + limits + a reaper for orphans.
- **Managed image lifecycle.** Fingerprinting, lease-based concurrency, crash recovery, registry pull/push — agent-cage has none.
- **Pluggable by design.** `IExecutionEnvironment` lets us add ecs/k8s/firecracker without touching callers.
- **Native coding-agent protocol.** The runner ships into the box and speaks structured JSON — typed results, not scraped terminal output.

What we can implement better

Their idea is right; their execution is a prototype. We can do it better because we already own the orchestration spine.

- **Egress policy as first-class config** — extend `network` to accept `{ mode:"allowlist", allow:[...], denyByDefault:true }`, per-sandbox / per-org.
- **Per-run proxy sidecar** (not one shared proxy) on a private network; inject `HTTP_PROXY` into the runner. Policy isolated per tenant.
- **DNS lockdown by default** — controlled resolver answers only allowlisted names; closes the IP-literal bypass.
- **Audit against the run model** — every blocked/allowed request keyed to `runId`, surfaced through existing admin routes (agent-cage can't — it has no run model).
- **Ship the hardening they only recommend** — non-root runner, dropped caps, seccomp, path to gVisor/Kata/Firecracker.

Field context

PRODUCT

ISOLATION

EGRESS GOVERNANCE

E2B	Firecracker microVM (own kernel, GPU)	none built-in
Daytona	Docker + gVisor	none built-in
agent-infra/sandbox	single container (seccomp off)	none
agent-cage	single container	strong
Journeyman	per-run container + limits, pluggable	gap

The whole field competes on compute isolation. Almost nobody ships egress governance — agent-cage is the outlier. Add a native egress layer and we'd pair orchestration leadership with a security capability E2B and Daytona lack out of the box.

Recommendation — which one to go with

GO WITH

Build the **native egress-governance layer**. Do **not** adopt or wrap agent-cage in production — it's a single-container prototype with no multi-tenancy, durable execution, or image lifecycle. Borrow its pattern; own the implementation.

- 1 Spike** — stand up agent-cage for an afternoon, study its `coding-agent` preset + mitmproxy rules. Keep the allowlist shape, discard the rest.
- 2 MVP** — per-run egress-proxy sidecar + `HTTP_PROXY` injection + allowlist config; default allow = package registries + configured git host.
- 3 Harden** — DNS lockdown, non-root runner, capability drops, seccomp.
- 4 Audit** — egress log keyed to `runId`, surfaced via existing admin routes.
- 5 Future** — Firecracker/gVisor backend for kernel-level isolation to match E2B/Daytona on the compute axis.

Sources: [pnnl/agent-cage](#) · [agent-infra/sandbox](#) · [Daytona vs E2B \(Northflank\)](#) · [E2B/Daytona/Firecracker \(Spheron\)](#) · Journeyman source: `packages/sandbox/src/backends/docker/`